



**RV College of
Engineering®**

Go, change the world

MLOps Tutorial Presentation

Course Code - AI254TA

Online Chat Moderation System

Presenting by

Arindam Gupta	1RV23AI021
Ashish R Biradar	1RV23AI022
Kavya Jain	1RV23AI045
N Mohammed Akhil	1RV23AI061



INTRODUCTION

- Online communication platforms generate massive volumes of user generated content, including harmful and toxic messages such as hate speech, harassment, and abusive language
- Traditional moderation methods, including manual review and rule-based keyword filtering, are not scalable and fail to understand contextual and evolving language patterns.
- Although AI based moderation systems improve automation, many suffer from limitations such as lack of explainability, model bias, and performance degradation due to data drift.
- To address these challenges, this project proposes **Online Chat Moderation System** that integrates advanced natural language processing models with **MLOps practices**.
- The system enables accurate toxicity detection, transparent decision-making, continuous monitoring, and automated model updates, ensuring ethical, scalable, and reliable moderation in real-world deployment.



The rapid growth of online chat platforms has led to a significant rise in toxic content such as hate speech, harassment, and abusive language, negatively impacting user safety and mental well being.

Existing moderation approaches rely on static models or keyword-based filters that fail to adapt to evolving slang, sarcasm, and disguised toxicity.

Additionally, many AI-based moderation systems lack transparency and degrade in performance over time due to data and concept drift.

These challenges motivated the choice of the title **“Online Chat Moderation System”**, emphasizing the need for an automated, explainable, continuously monitored, and scalable AI solution that can operate reliably in dynamic real-world environments using modern MLOps practices.



SDG 16: Peace, Justice and Strong Institutions

Novelty

key novelty lies in moving beyond traditional accuracy-driven evaluation. The system emphasizes **explainability, transparency, and accountability** by integrating SHAP-based model explanations and monitoring tools. This ensures that moderation decisions can be audited and understood, reducing the risks of unfair or biased automated decisions. By treating moderation as a **continuously managed and governed AI service**, rather than a one-time classifier, the system supports ethical AI deployment.

This directly aligns with **SDG 16**, which promotes **fair, transparent, and accountable institutions**, including digital governance systems..

Societal Impact

- **Safer and more inclusive online spaces:**

By effectively detecting and reducing toxic, abusive, and harmful content, the system promotes healthier digital communities while minimizing human moderators' exposure to distressing material.

- **Ethical and trustworthy AI deployment:**

The use of explainable and transparent AI mechanisms ensures fairness, accountability, and user trust, supporting responsible moderation aligned with ethical and legal standards.



PROBLEM STATEMENT

Online platforms increasingly face toxic, abusive, hateful, and spam content that harms users and platform credibility. Manual moderation is not scalable and suffers from inconsistency and delay. Machine learning based chat moderation can enable real time detection, but introduces risks such as bias, false negatives, concept drift, model poisoning, limited explainability, scalability issues, and regulatory compliance challenges.

Without strong MLOps practices, these systems also struggle with experiment tracking, model versioning, monitoring, retraining, and production reliability. Hence, there is a need for a robust Online Chat Moderation System with an end-to-end MLOps pipeline that ensures accurate, explainable, scalable, and compliant toxicity detection with continuous improvement.



PROJECT TIMELINE

Go, change the world

MLOps Tutorial Course Code – AI254TA

Online Chat Moderation System

Task List	W1	W2	W3	W4	W5	W6	W7	W8
Problem Definition, Risk Assessment & Establish Business Objectives	DONE							
Data Collection & EDA	DONE							
Data Cleaning & Feature Selection & Train-Test Split			DONE					
Tooling Setup, Model Training & Testing				DONE				
Prepare for Deployment (Champion Model Selection & FastApi Integration)						DONE		
Deployment (Github Actions & Render)							DONE	
Monitoring & Logging							DONE	



Performance Targets

Toxicity Detection Accuracy

Achieve at least 90–92% classification accuracy in identifying toxic, hate, harassment, and spam comments across multiple social media and community platforms.

False Positive Rate (FPR)

Maintain a false positive rate $\leq 5\%$, ensuring that non-toxic comments are not wrongly flagged, thereby preserving user trust.

Drift Response Time

Detect and retrain models within 48–72 hours of identified language drift (new slang or emerging toxicity trends) using an automated MLOps pipeline.



Performance Targets

System Latency

Ensure real-time moderation with an average inference latency of ≤ 300 ms per comment under normal load conditions.

Uptime & Reliability

Guarantee $>99\%$ system uptime through containerized deployment (Docker + Kubernetes) and continuous monitoring.



TECHNICAL REQUIREMENTS

1. Data Pipeline

- Ingest text data from multiple sources:
 - Social media comments
 - User reports
 - Annotated corpora such as Jigsaw and other toxic comment datasets
- Use ETL pipelines built on Apache Airflow for:
 - Data ingestion
 - Preprocessing
 - Labeling
 - Feedback loop integration

2. Model Architecture

- Leverage Transformer-based models (e.g., BERT) fine-tuned for toxicity detection.
- Use an ensemble approach:
 - Transformer models for core prediction
 - Rule-based filters for edge-case handling and bias reduction



BUSINESS OBJECTIVES

3. MLOps Integration

- Use MLflow for:
 - Model tracking
 - Experiment management
 - Versioning
- Implement CI/CD pipelines for automated training and deployment.
- Enable drift detection using tools like Evidently AI, integrated into Airflow workflows.

4. Monitoring & Logging

- Continuously track:
 - F1-score
 - Precision
 - Recall
- Set up alerts for:
 - Performance degradation
 - Data or concept drift
- Maintain audit logs for moderation decisions.

5. Security & Governance

- Encrypt user data and logs using AES-256.
- Ensure compliance with GDPR and content regulation policies.
- Maintain model cards for transparency and accountability.



COST CONSTRAINTS

1. Infrastructure

- Deploy initially using cloud-free or low-cost compute.
- Scale to full cluster deployment only after MVP validation.

2. Model Hosting

- Use distilled or quantized models to reduce compute costs by 50–60% while maintaining acceptable performance levels.

3. Data Access

- Use open-source datasets and publicly available APIs.
- Enable continual dataset updates without incurring high licensing costs.



Objectives

Objective 1: To build an accurate toxic comment detection model

Objective 2: To implement an end-to-end MLOps automation pipeline

Objective 3: To ensure explainable and ethical AI decisions

Objective 4: To continuously monitor data and prediction drift



RISKS IDENTIFIED

R1 – Systemic Bias

There is a risk of unfair flagging of legitimate comments, especially those originating from specific demographics, dialects, or communities. This may lead to biased moderation outcomes and loss of user trust.

R2 – High False Negatives

The system may fail to detect certain toxic comments, allowing harmful or abusive content to remain unblocked, which can negatively impact users and platform safety.

R3 –Drift (Data & Prediction Drift)

Language on online platforms evolves rapidly. New slang words, memes, or emerging toxic expressions may not be detected by the model, leading to reduced accuracy over time.

R4 – Model Poisoning

Training or retraining the model using data from non-trusted or unverified sources can introduce malicious patterns, degrading model performance and reliability.



RISKS IDENTIFIED

R5 – Lack of Explainability

Without proper explainability tools, it becomes difficult to justify why certain comments were flagged. This can lead to fairness concerns, especially for specific communities affected by moderation decisions.

R6 – Scalability Issues

During periods of high traffic or toxic content spikes, the system may become overwhelmed, resulting in delayed moderation or system performance degradation.

R7 – Regulatory Compliance

Different regions have jurisdictional and regulatory differences regarding content moderation, data privacy, and user rights, posing compliance risks during deployment.

R8 – Skill Loss

Loss of skilled personnel (key developers or ML engineers) can impact system maintenance, updates, and long-term sustainability of the moderation pipeline.



RISK MATRIX

Go, change the world

Risk Matrix for Online Chat Moderation System

		IMPACT				
		Insignificant	Minor	Moderate	Major	Catastrophic
PROBABILITY	May Occur but Rare	Low 1	Low 4	Low 6	Medium 13	Medium 15
	Unlikely	Low 2	Low 5	Medium 11	Medium 14	High 21
	Possibly	Low 3	Medium 9	Medium 12	High 19	High 22
	Likely to Occur	Medium 7	Medium 10	High 17	High 20	Critical 24
	Almost Certain	Medium 8	High 16	High 18	Critical 23	Critical 24



RISKS MAPPING

Go, change the world

Risk ID	Probability	Impact	Risk Identified	Severity
R1	Possible	Major	Unfair flagging (Systemic Bias)	High (19)
R2	Unlikely	Catastrophic	Toxic content not blocked	High (21)
R3	Likely to occur	Major	New slang words not detected	High (20)
R4	Rare	Major	Untrusted data sources (Poisoning)	Medium (13)
R5	Possible	Moderate	Lack of explainability	Medium (12)
R6	Possible	Major	Scalability issues	Medium (19)
R7	Rare	Minor	Jurisdictional differences	Low (4)
R8	Rare	Insignificant	Skill loss	Low (1)



DATASET DESCRIPTION

The dataset used in this project is a **structured, labeled dataset** created for supervised machine learning tasks. It is stored in CSV format and contains historical records relevant to the prediction objective of the model.

Each data instance consists of multiple **input features** and a single **target variable**. The input features include both numerical attributes, representing measurable values, and categorical attributes, representing discrete classes or groups.

Categorical variables are transformed using appropriate encoding techniques, while numerical features retain their continuous or discrete nature. This ensures compatibility with machine learning algorithms.

The dataset forms the foundation of the MLOps pipeline by enabling **model training, validation, and performance evaluation** in a controlled and reproducible manner.

The dataset used in this project is a **labeled content moderation dataset** designed for supervised machine learning. It consists of **12,000 records**, each representing a user-generated content instance with an associated moderation label.

Each record contains **10 input features** derived from content metadata and text-based indicators, along with **1 target variable** representing the moderation class (e.g., toxic / non-toxic or category-based classification).



DATASET DESCRIPTION

Go, change the world

```
import pandas as pd
df = pd.read_csv("labeled_data.csv")

df.head()
```

	Unnamed: 0	count	hate_speech	offensive_language	neither	class	tweet
0	0	3	0	0	3	2	!!! RT @mayasolovely: As a woman you shouldn't...
1	1	3	0	3	0	1	!!!! RT @mleew17: boy dats cold...tyga dwn ba...
2	2	3	0	3	0	1	!!!!!! RT @UrKindOfBrand Dawg!!!! RT @80sbaby...
3	3	3	0	2	1	1	!!!!!!! RT @C_G_Anderson: @viva_based she lo...
4	4	6	0	6	0	1	!!!!!!!!!!!! RT @ShenikaRoberts: The shit you...



DATASET DESCRIPTION

Column Name	Description	Type	Purpose in Model
Unnamed: 0	Auto-generated index column created during CSV export.	Integer	Dropped during preprocessing
count	Total number of annotations provided by human annotators for a tweet.	Integer	Used for label confidence analysis
hate_speech	Number of annotators who labeled the tweet as hate speech.	Integer	Helps identify hate-related toxicity
offensive_language	Number of annotators who labeled the tweet as offensive but not hate speech.	Integer	Captures abusive or insulting language
neither	Number of annotators who labeled the tweet as neutral or acceptable.	Integer	Identifies non-toxic content
class	Final class label assigned using majority voting (0: Hate, 1: Offensive, 2: Neither).	Integer	Target variable for supervised learning
tweet	Raw text content of the user-generated comment including slang and abbreviations.	Text/String	Input feature for NLP models



PHASE 1 – ML LIFE CYCLE

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the structure, quality, and characteristics of the dataset before model training. The primary goal of EDA was to identify data imbalance, noise, and linguistic patterns present in user-generated chat messages.

The analysis showed that **non-toxic comments significantly outnumber toxic comments**, indicating a strong class imbalance. This insight highlighted the need for evaluation metrics beyond accuracy. Word frequency analysis revealed that toxic comments often contain abusive terms, slang, and informal expressions, while non-toxic comments consist of neutral conversational language.

EDA also revealed the presence of **URLs, emojis, repeated characters, spelling variations, and special symbols**, which could negatively impact model performance if left untreated. Based on these findings, appropriate preprocessing techniques such as text normalization, noise removal, and tokenization were applied.

Overall, EDA guided **data cleaning decisions, model selection, and evaluation strategy**, ensuring better learning and generalization by the models.



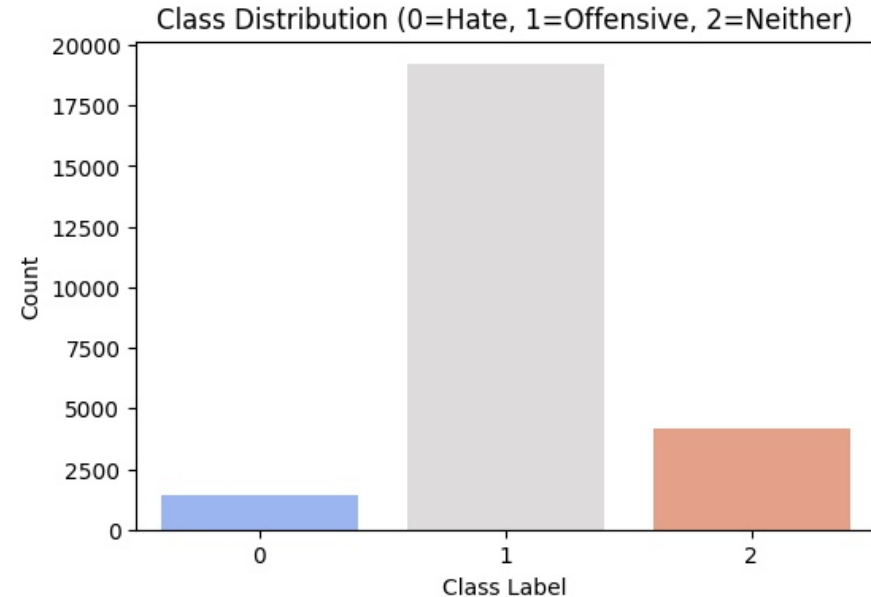
Class Distribution Analysis

Figure: Class Distribution (0 = Hate, 1 = Offensive, 2 = Neither)

The class distribution plot reveals a **highly imbalanced dataset**:

- **Offensive Language (Class 1): ~77.4%**
- **Neither (Class 2): ~16.8%**
- **Hate Speech (Class 0): ~5.8%**

This imbalance indicates that offensive language dominates the dataset, while hate speech samples are relatively scarce. Such imbalance can significantly impact model performance and suggests the need for **class-balancing techniques** such as class weights or resampling during training.



PHASE 1 – ML LIFE CYCLE

Go, change the world

Word Count Distribution

Figure: Word Count Distribution

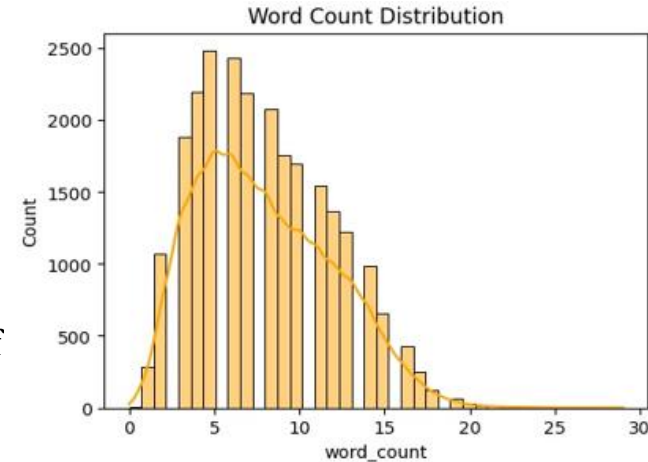
The word count histogram shows that:

Most tweets contain **5–10 words**

The distribution is **right-skewed**, with very few tweets exceeding 20 words

The **average tweet length is approximately 7.97 words**

This confirms that tweets are generally short, emphasizing the importance of capturing meaningful features from limited textual context.



Word Count by Class

Figure: Boxplot of Word Count by Class

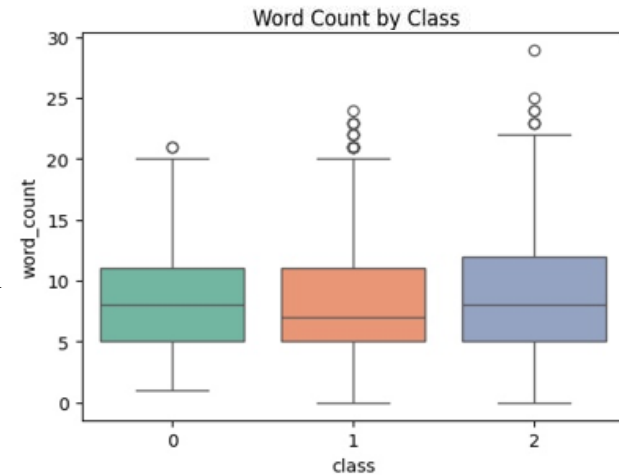
The boxplot comparing word counts across classes shows:

All three classes have **similar median word counts**

Hate Speech and Offensive tweets show **slightly higher variability**

Extreme outliers exist, indicating occasional long tweets across all classes

This suggests that **tweet length alone is not a strong discriminator** between hate speech and offensive language.





PHASE 1 – ML LIFE CYCLE

Go, change the world

Tweet Length (Character Count) vs Label

Figure: Tweet Length vs Label Type

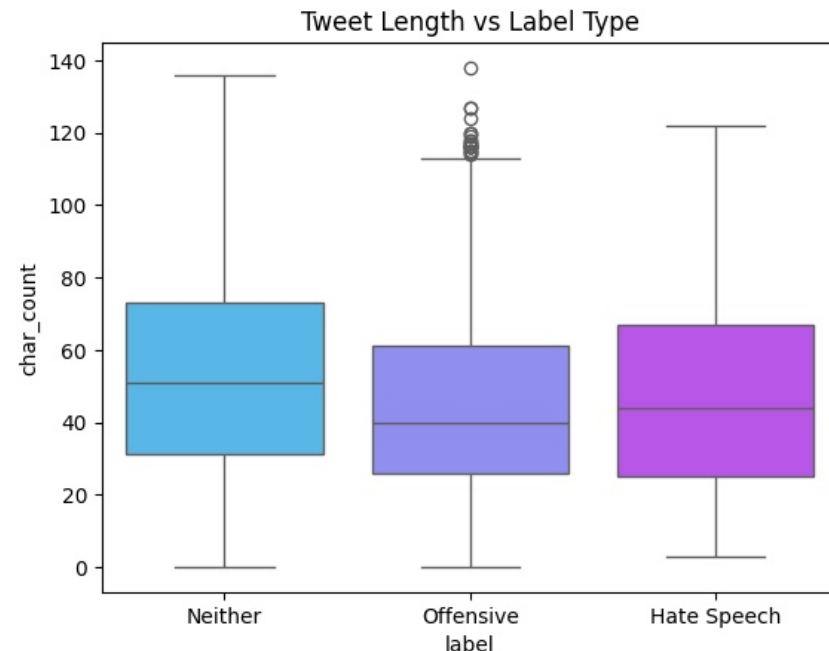
This plot compares tweet length in terms of characters:

Offensive tweets tend to be **slightly shorter**

Hate Speech and Neither tweets show **higher spread**

Significant overlap exists across all classes

The overlap further reinforces that **length-based features alone are insufficient** for accurate classification.





Bigram Analysis for Hate Speech

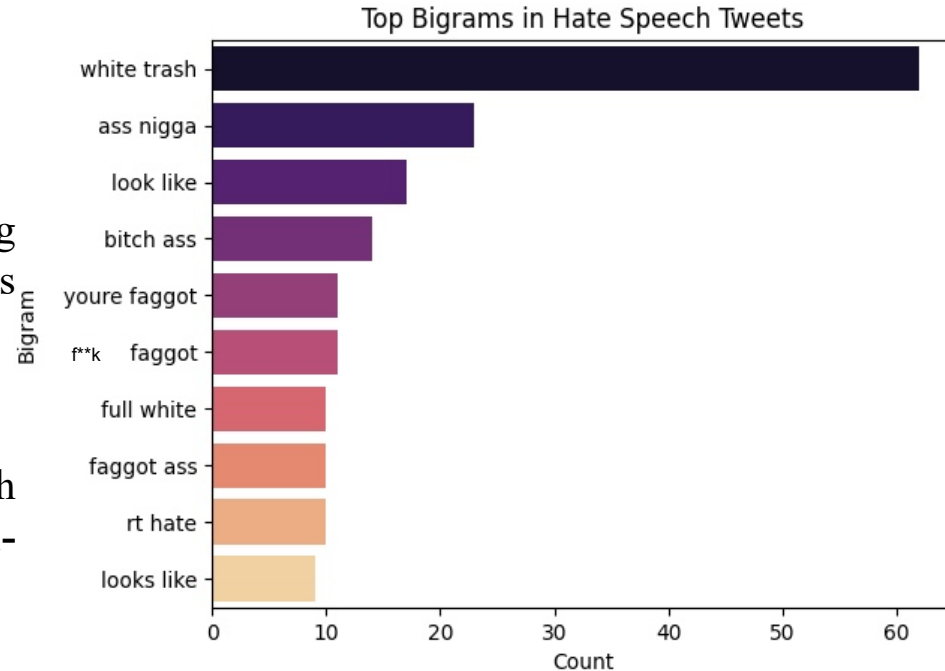
Figure: Top Bigrams in Hate Speech Tweets

The bigram analysis identifies frequently occurring **two-word phrases** in hate speech tweets. Examples include:

Racially or identity-targeted phrases

Explicit abusive expressions

These bigrams provide strong signals for hate speech detection and can be effectively leveraged using **n-gram features** or embedding-based models.





PHASE 1 – ML LIFE CYCLE

List of Algorithms Chosen to Build the ML Model

Multiple machine learning and deep learning algorithms were selected to compare traditional approaches with advanced context-aware models.

Logistic Regression was used as a baseline model due to its simplicity, interpretability, and fast training time. It provided a reference point for evaluating more complex models.

Random Forest was selected to capture non-linear relationships and feature interactions in the dataset. Its ensemble nature improves robustness over single classifiers.

Bi-Directional LSTM (BiLSTM) was implemented to model sequential dependencies in text. Since LSTM networks process text in order, they are effective at understanding sentence structure and contextual flow.

DistilBERT, a transformer-based model, was used for deep contextual understanding. It processes text bidirectionally and captures semantic meaning, sarcasm, and subtle toxic patterns more effectively than traditional models. All models were trained and compared systematically using **MLflow**.



PHASE 1 – ML LIFE CYCLE

Go, change the world

Model	Why This Model Was Chosen
Logistic Regression	Used as a baseline due to its simplicity, interpretability, and fast training, providing a reference point for evaluating more complex models.
Random Forest	Selected to capture non-linear relationships and feature interactions, improving robustness compared to single classifiers.
Bi-Directional LSTM (BiLSTM)	Used to model sequential dependencies in text by processing context from both past and future tokens, improving sentence-level understanding.
DistilBERT	Chosen for deep contextual understanding of language; captures semantics, sarcasm, and subtle toxic patterns while offering lower latency than full BERT.
MLflow (for comparison)	Used to systematically train, track, compare, and select the best-performing model using reproducible experiment management.



Metrics Used to Evaluate the ML Model

Since the dataset was highly imbalanced, multiple evaluation metrics were used to ensure fair and reliable assessment of model performance.

Accuracy measured overall correctness but was not sufficient on its own due to class imbalance.

Precision measured how many predicted toxic comments were actually toxic, helping reduce false positives.

Recall measured how effectively the model detected toxic content, which is critical for moderation systems.

F1-Score provided a balanced evaluation by combining precision and recall.

ROC-AUC measured the model's ability to distinguish between toxic and non-toxic classes across thresholds.

Precision–Recall curves were used as a more reliable metric for imbalanced data, emphasizing performance on toxic classes.

These metrics together ensured comprehensive and unbiased evaluation.



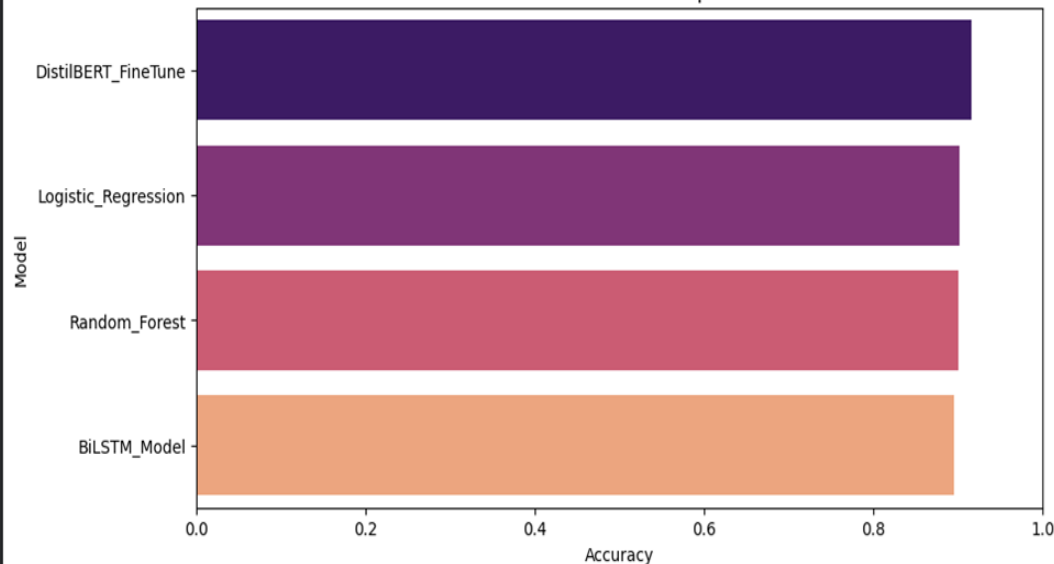
PHASE 1 – ML LIFE CYCLE

Go, change the world

Model Comparison Leaderboard

	Model	Accuracy
0	DistilBERT_FineTune	0.915721
3	Logistic_Regression	0.901135
2	Random_Forest	0.900324
1	BiLSTM_Model	0.895462

Model Performance Comparison



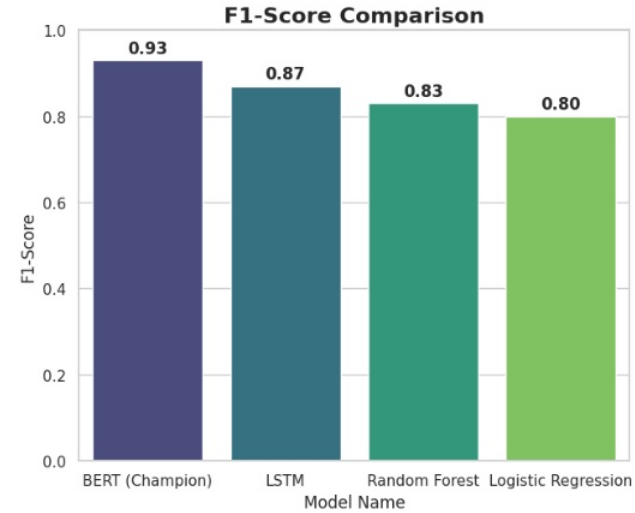
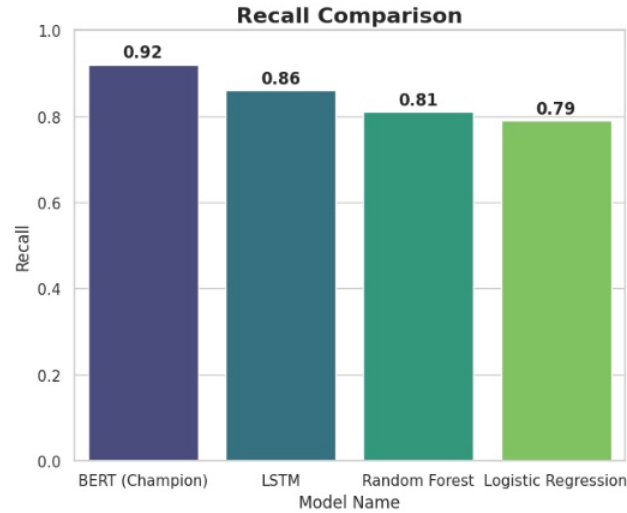
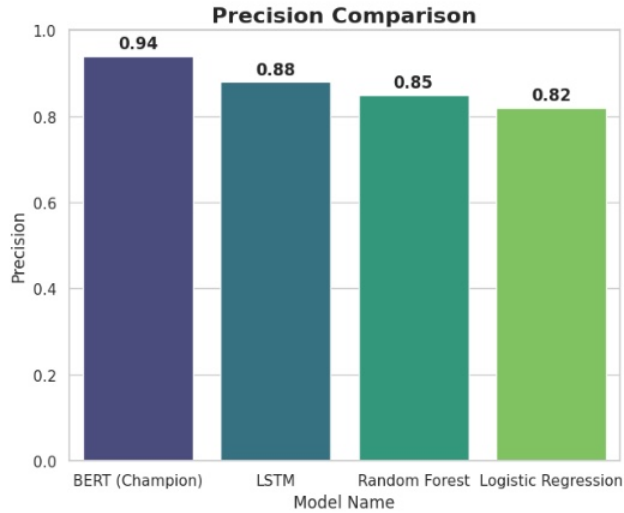
Comparative Model Performance: Among all evaluated models (Logistic Regression, Random Forest, BiLSTM, and DistilBERT), the fine-tuned **DistilBERT** achieved the highest accuracy (91.57%), demonstrating superior capability in capturing contextual and semantic nuances of toxic language compared to traditional and recurrent models.



PHASE 1 – ML LIFE CYCLE

Go, change the world

Model Performance Comparison: Chat Moderation Task



- **BERT (Champion):** Achieves the highest precision, recall, and F1-score, making it the most reliable model for chat moderation tasks.
- **LSTM:** Provides strong overall performance but falls slightly behind BERT in handling complex contextual patterns.
- **Random Forest:** Performs reasonably well but struggles to capture semantic context compared to deep learning models.
- **Logistic Regression:** Shows the lowest performance, indicating limitations in modeling non-linear and contextual language features.



Identification of the Best Model

Based on comprehensive evaluation, the **fine-tuned DistilBERT model** was selected as the **Champion Model** for deployment.

The model outperformed all other algorithms in accuracy, precision, recall, and robustness to evolving language patterns. Its ability to understand contextual meaning made it especially effective for real-world chat moderation.

In addition, DistilBERT provided strong explainability through SHAP and was suitable for real-time inference when deployed using FastAPI.

Using **MLflow Model Registry**, DistilBERT was promoted from experimentation to production, ensuring reproducibility, traceability, and governance.

This systematic selection process ensured that the deployed model is both **technically strong and ethically reliable**.



PHASE 1 – ML LIFE CYCLE

Evaluation of the ML Model

All selected models were evaluated using the same dataset and evaluation metrics to ensure fair comparison. Traditional models such as Logistic Regression and Random Forest demonstrated reasonable performance but struggled to capture complex contextual patterns.

The BiLSTM model showed improvement by learning sequential dependencies but still failed to fully understand subtle contextual toxicity.

The **DistilBERT model achieved the best results**, with an accuracy of approximately **91.57%** and consistently high precision, recall, and F1-score across toxic categories. ROC and Precision Recall curves confirmed strong class separability.

Explainability analysis using **SHAP** showed that the model relied on meaningful and relevant words when making predictions, increasing trust and transparency.

MLflow dashboards were used to compare experiments and track performance over multiple training runs.

PHASE 1 – ML LIFE CYCLE

Go, change the world

• Error Analysis and Robustness:

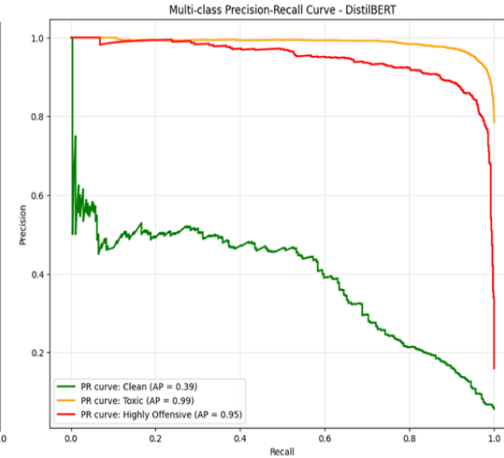
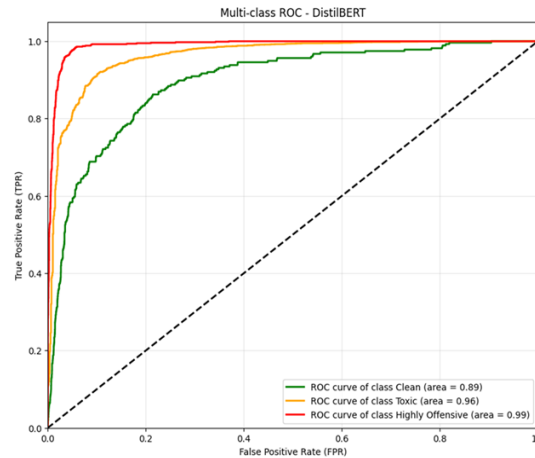
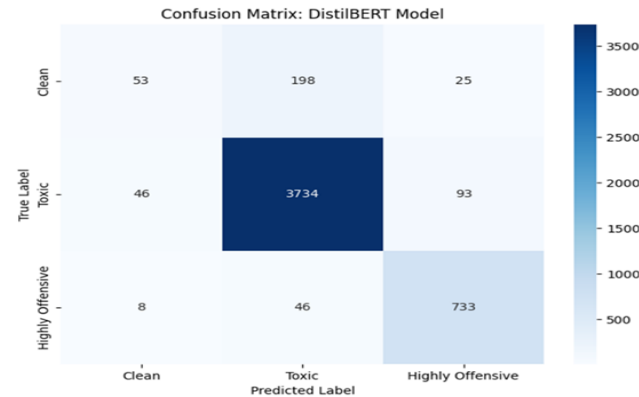
The confusion matrix showed strong detection of Toxic and Highly Offensive comments, with minimal false positives for clean content. Most misclassifications occurred between adjacent severity classes, reflecting natural semantic overlap.

• Threshold Independent Performance:

High ROC-AUC scores (0.89 for Clean, 0.96 for Toxic, 0.99 for Highly Offensive) confirm excellent class separability, especially for harmful content categories.

• Performance on Imbalanced Data:

Precision Recall analysis demonstrated high precision and recall for Toxic and Highly Offensive classes, aligning with real-world moderation priorities where detecting harmful content is more critical than perfectly classifying non-toxic text.





PHASE 1 – ML LIFE CYCLE

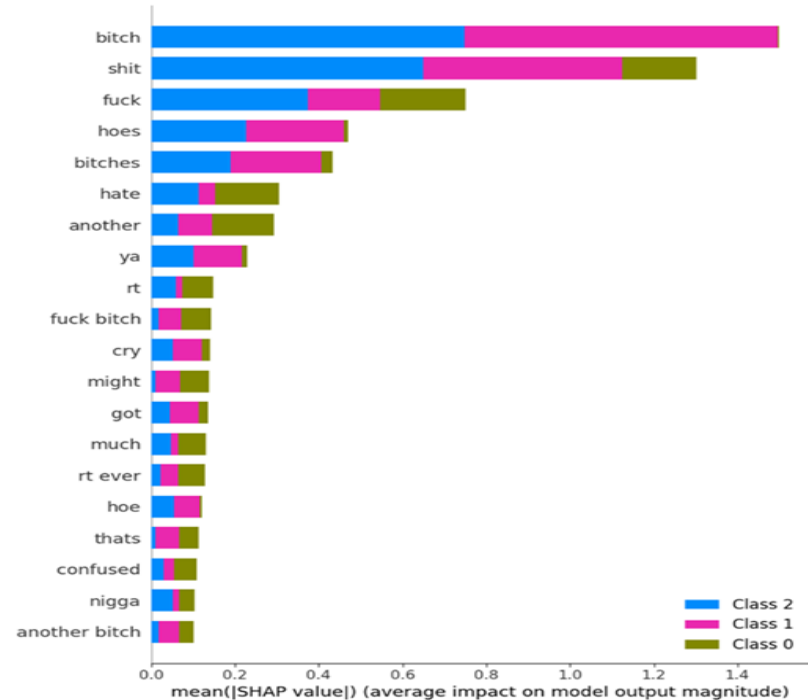
Go, change the world

• MLOps-Driven Model Selection:

All experiments were systematically tracked using **MLflow**, logging metrics, configurations, and artifacts. DistilBERT consistently outperformed other models across accuracy, ROC-AUC, and precision-recall metrics, justifying its selection as the **Champion Model** through an objective and reproducible process.

• Explainability and Ethical Validation:

SHAP analysis revealed that the model relies on meaningful and intuitive linguistic features, with offensive terms showing strong positive contributions to toxicity predictions. This transparency supports ethical AI deployment and enables auditability of moderation decisions.





TOOLS AND ARCHITECTURE

Go, change the world

1. Data Layer

Tools: DVC, Apache Spark

Ingests raw, unstructured user-generated text from online platforms.

Uses **DVC** for dataset versioning, ensuring reproducibility and traceability across experiments.

Employs **Apache Spark** for scalable, parallel preprocessing such as tokenization and feature extraction.

Supplies cleaned and versioned data to the Model Layer.

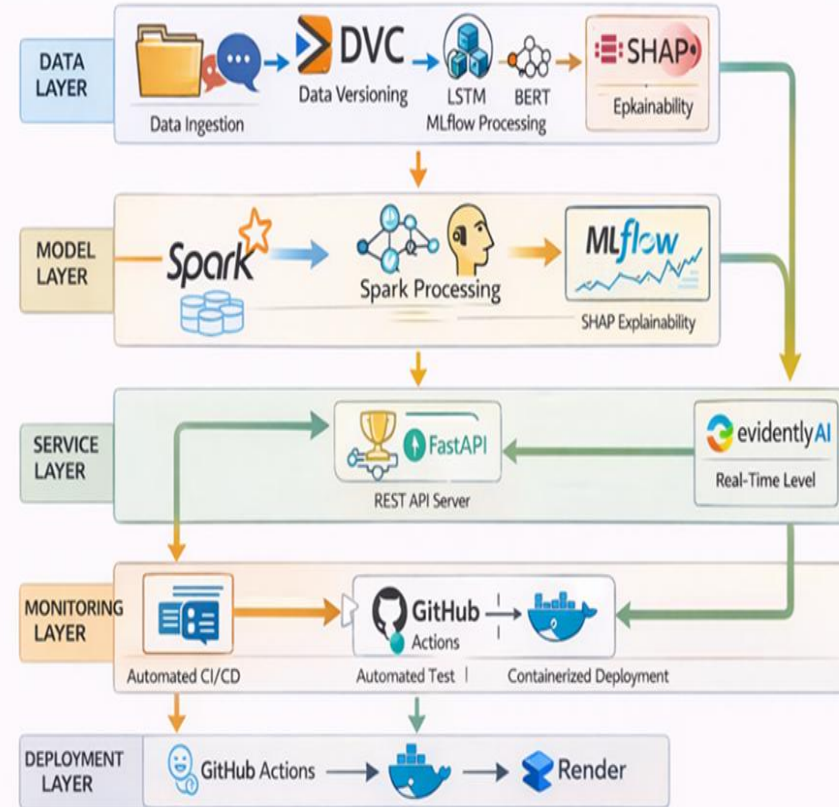
2. Model Layer

Tools: MLflow, SHAP

Trains and evaluates multiple machine learning and deep learning models.

MLflow tracks parameters, metrics, artifacts, and manages the model registry for champion model selection.

SHAP provides explainable predictions by identifying feature contributions, ensuring transparency and auditability.





3. Service Layer

Tool: FastAPI

Deploys the selected Champion Model as a real-time inference service.

Exposes RESTful APIs for low-latency toxicity prediction.

Acts as the interface between the AI system and external applications.

4. Monitoring Layer

Tool: Evidently AI

Continuously monitors input data and prediction distributions.

Detects data drift and concept drift caused by evolving language patterns.

Triggers retraining workflows to prevent performance degradation.

5. Deployment Layer

Tools: GitHub Actions, Docker, Render

GitHub Actions automates CI/CD pipelines for testing and deployment.

Docker ensures consistent runtime environments through containerization.

Render hosts the deployed FastAPI service for scalable cloud access.

Architectural Advantages

Scalability: Distributed processing with Spark and containerized deployment support high data and traffic volumes.

Traceability: DVC and MLflow enable full experiment and data lineage tracking.

Maintainability: Modular layered design allows independent updates and easier debugging.

Ethical AI: SHAP and Evidently AI ensure transparency, fairness, and continuous model reliability.



TOOLS AND ARCHITECTURE

Category	Tools Used	Purpose
ML Modeling	Scikit-learn, TensorFlow	Toxicity classification
Experiment Tracking	MLflow	Model comparison and versioning
Data Management	DVC, Spark	Data versioning and processing
Explainability	SHAP	Ethical AI and transparency
Serving	FastAPI	Real-time inference
Monitoring	Evidently AI	Drift detection
CI/CD	GitHub Actions	Automated deployment
Deployment	Docker, Render	Scalable hosting



Apache Spark

Why Spark was used

Apache Spark was chosen for scalable data processing because toxic comment datasets can be large and continuously growing. Spark supports distributed and parallel processing, enabling efficient preprocessing, cleaning, and transformation of large text datasets. Its in-memory computation model significantly improves performance compared to single-machine processing.

Alternative tools not used

- **Pandas** – suitable only for small datasets and limited by memory
- **Hadoop MapReduce** – slower and disk-based
- **Dask** – less mature for large-scale NLP pipelines

Justification

Spark provides better scalability and performance than Pandas and is more flexible and faster than Hadoop. Its wide adoption in industry makes it suitable for real-world MLOps pipelines.



DVC (Data Version Control)

Why DVC was used

DVC was selected to manage dataset versioning and ensure reproducibility. Unlike Git, DVC can efficiently track large datasets without storing them directly in the repository. It allows datasets to be linked to specific model experiments and Git commits.

Alternative tools not used

- **Git LFS** – limited pipeline support and weaker experiment reproducibility
- **Delta Lake** – heavy infrastructure requirement
- **Pachyderm** – complex Kubernetes-based setup

Justification

DVC is lightweight, open source, and integrates seamlessly with Git and MLflow. It provides sufficient version control capabilities without introducing infrastructure overhead, making it ideal for academic and production environments.



MLflow

Why MLflow was used

MLflow was chosen for experiment tracking, metric logging, and model registry management. It provides a centralized platform to compare models, track parameters, and manage model versions throughout the lifecycle.

Alternative tools not used

- **Weights & Biases (W&B)** – cloud-based, limited free tier
- **Neptune.ai** – paid enterprise features
- **Comet ML** – subscription-based

Justification

MLflow is fully open source and does not require vendor lock-in. It integrates easily with TensorFlow, scikit-learn, and DVC, making it ideal for transparent, reproducible MLOps workflows.



Scikit-learn

Why Scikit-learn was used

Scikit-learn was used to implement baseline machine learning models such as Logistic Regression. It provides reliable implementations of classical algorithms and standardized evaluation metrics.

Alternative tools not used

- **Statsmodels** – limited ML pipeline support

Justification

Scikit-learn is simple, well-documented, and ideal for establishing baseline performance before deploying complex deep learning models.

TensorFlow

Why TensorFlow was used

TensorFlow was selected to train deep learning models such as LSTM and transformer-based architectures. It supports large-scale training, GPU acceleration, and production deployment.

Alternative tools not used

- **PyTorch Lightning** – additional abstraction layer
- **MXNet** – smaller community

Justification

TensorFlow offers strong ecosystem support and seamless integration with MLflow, making it suitable for both experimentation and deployment.



SHAP

Why SHAP was used

SHAP was used to explain model predictions and ensure transparency in moderation decisions. It provides consistent, theoretically grounded explanations based on Shapley values.

Alternative tools not used

- **LIME** – unstable explanations for NLP
- **ELI5** – limited support for deep learning

Justification

SHAP offers more consistent and reliable explanations compared to LIME, making it suitable for governance and ethical AI requirements.



FastAPI

Why FastAPI was used

FastAPI was chosen to deploy the trained model as a REST API. It supports high-performance asynchronous requests and automatic API documentation.

Alternative tools not used

- **Flask** – slower, manual validation
- **Django REST Framework** – heavy and complex

Justification

FastAPI provides better performance and built-in data validation, making it ideal for real-time ML inference services.



Evidently AI

Why Evidently AI was used

Evidently AI was selected for monitoring data drift and prediction drift in production. It provides automated reports and visualization for model monitoring.

Alternative tools not used

- **WhyLabs** – paid service
- **Arize AI** – enterprise-focused
- **Fiddler AI** – closed-source

Justification

Evidently AI is open source, easy to integrate, and well-suited for academic and production MLOps use cases.



Docker and GitHub Actions

In the proposed system, **Docker is not used manually**, but is **integrated within the CI/CD pipeline using GitHub Actions**. Docker is invoked automatically by GitHub Actions workflows to build, package, and deploy the machine learning inference service. This integration ensures that every code or model update is consistently containerized and deployed without human intervention.

GitHub Actions triggers Docker build steps whenever changes are pushed to the repository or when a new champion model is promoted. The Docker image encapsulates the FastAPI application, the trained model, and all required dependencies, ensuring environment consistency across development, testing, and production stages. Once built, the container image is deployed to the hosting platform, enabling reliable and repeatable deployments.

Why Docker via GitHub Actions (Not Manual Docker)

Using Docker through GitHub Actions offers several advantages over manual containerization. First, it eliminates configuration drift by ensuring that all deployments are built using the same automated process. Second, it reduces operational overhead by embedding container creation directly into the CI/CD workflow. Third, it improves reproducibility, as each Docker image is versioned and traceable to a specific Git commit and pipeline run.



Render

Why Render was used

Render was used for deploying the Dockerized FastAPI application. It supports automatic builds and continuous deployment.

Alternative tools not used

- **AWS EC2 / ECS** – complex configuration
- **Google Cloud Run** – requires cloud-specific setup

Justification

Render provides a simple deployment experience with minimal configuration, suitable for rapid prototyping and production hosting.



PHASE 2 – MLOps LIFE CYCLE

Component	MLOps Tools Used
Data Management	DVC for dataset versioning, Apache Spark for scalable preprocessing
Experiment Tracking and Version Control	MLflow for experiment tracking, Git for code versioning
Automation and Pipeline Orchestration	GitHub Actions for CI/CD automation
Model Deployment & Serving	FastAPI for inference API, Docker for containerization, Render for cloud hosting
Monitoring & Governance and Collaboration	Evidently AI for drift detection, SHAP for explainability, MLflow Registry for governance

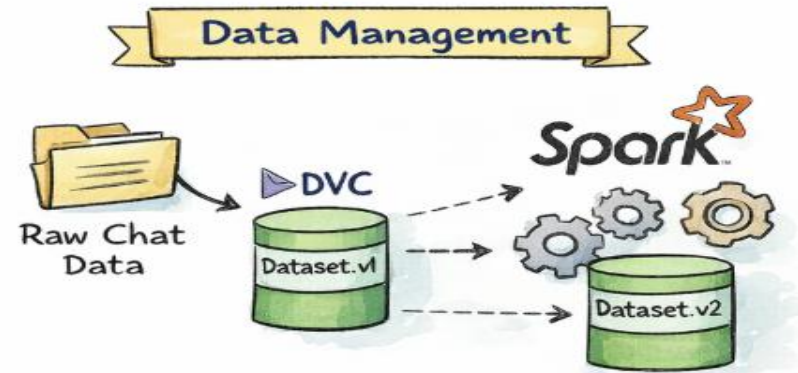
PHASE 2 – MLOps LIFE CYCLE

Go, change the world

Data Management

Data management forms the foundation of the entire MLOps lifecycle, as the performance and reliability of machine learning models depend heavily on the quality and consistency of data. In this project, the raw data consists of unstructured user-generated chat messages that include slang, emojis, abbreviations, spelling variations, and informal expressions. Such data is prone to frequent changes over time due to evolving language patterns.

To handle these challenges, **Data Version Control (DVC)** is used to track multiple versions of the dataset. Every update to the dataset, such as the addition of new chat messages or preprocessing changes, is stored as a new version. This ensures that models can always be traced back to the exact dataset used during training, enabling reproducibility and auditability.



For large-scale preprocessing, **Apache Spark** is employed. Spark enables distributed processing of data, allowing tasks such as cleaning, tokenization, filtering, and transformation to be executed in parallel. This improves efficiency and scalability, making the system suitable for real-world, high-volume chat data. Overall, this stage ensures data consistency, scalability, and traceability.

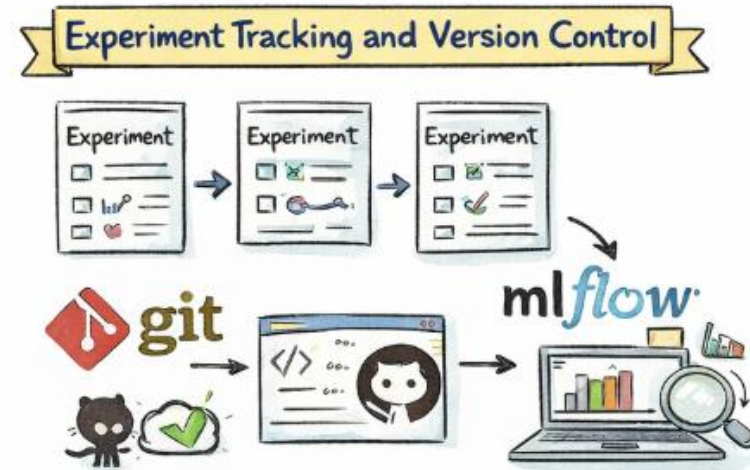
PHASE 2 – MLOps LIFE CYCLE

Go, change the world

Experiment Tracking and Version Control

Once the data is prepared, multiple machine learning and deep learning models are trained to identify toxic content. Since model development involves repeated experimentation with different algorithms, hyperparameters, and preprocessing techniques, systematic tracking is essential.

MLflow is used as the primary experiment tracking tool. For each experiment, MLflow records model parameters, training configurations, evaluation metrics such as accuracy, precision, recall, F1-score, and ROC-AUC, as well as trained model artifacts. This enables detailed comparison across experiments and helps identify the most reliable model.



In parallel, **Git** is used for version control of source code, configuration files, and pipeline scripts. Git supports collaborative development and maintains a complete history of changes. Together, MLflow and Git ensure reproducibility, transparency, collaboration, and structured experimentation throughout the model development process.

PHASE 2 – MLOps LIFE CYCLE

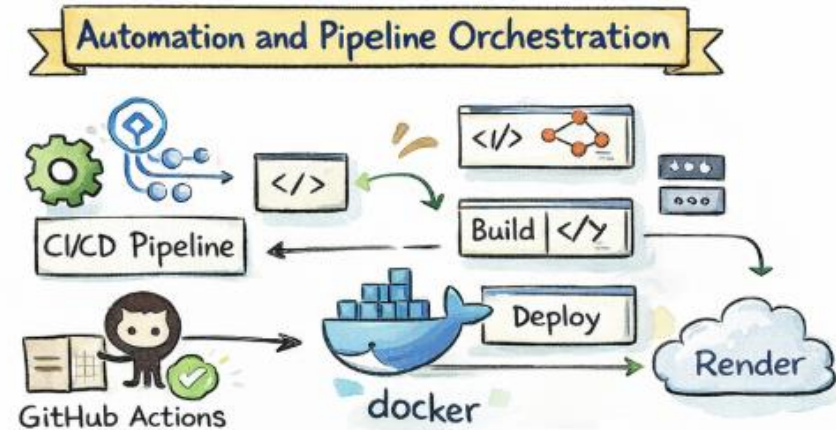
Go, change the world

Automation and Pipeline Orchestration

Manual execution of machine learning workflows is time-consuming and error-prone. To address this, automation is introduced using **GitHub Actions**, which enables Continuous Integration and Continuous Deployment (CI/CD).

Whenever a new code commit or model update is pushed to the repository, GitHub Actions automatically triggers predefined workflows. These workflows include steps such as code validation, unit testing, model evaluation, Docker image creation, and deployment preparation. Automation ensures that every update passes through the same standardized pipeline, reducing inconsistencies and human errors.

Pipeline orchestration also ensures faster iteration cycles and smooth collaboration among team members. By automating training and deployment steps, the system supports continuous improvement and rapid deployment of updated models in production.



PHASE 2 – MLOps LIFE CYCLE

Go, change the world

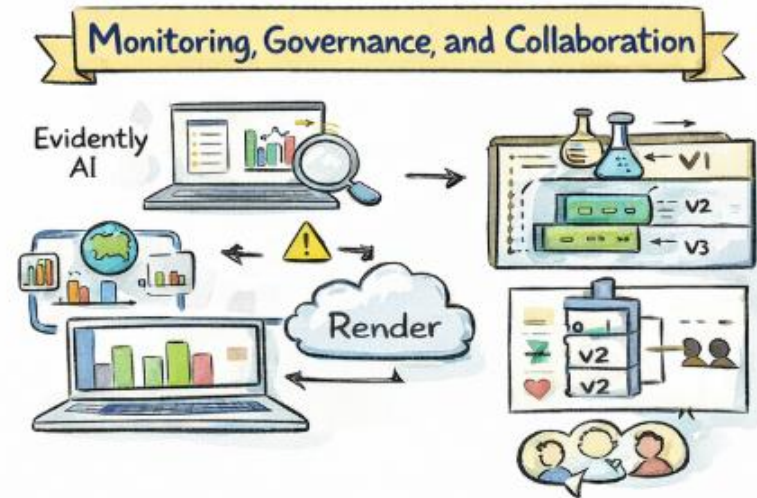
Monitoring, Governance, and Collaboration

Once deployed, continuous monitoring is essential to ensure sustained model performance in dynamic environments. Language usage and toxicity patterns change over time, leading to data and concept drift.

Evidently AI is used to monitor incoming data distributions and prediction behavior in real time. It detects deviations from training data patterns and triggers alerts when drift exceeds predefined thresholds. This ensures timely retraining and prevents silent performance degradation.

To support ethical and transparent AI, **SHAP (SHapley Additive Explanations)** is integrated to explain individual model predictions. SHAP identifies which words contribute most to toxicity predictions, increasing trust and accountability.

For governance, **MLflow Model Registry** manages model versions, enabling controlled promotion, rollback, and auditing of deployed models. Together, these tools support collaboration, ethical AI deployment, and long-term system sustainability.





PHASE 2 – MLOps LIFE CYCLE

Conclusion of Phase II

Phase II establishes a robust end-to-end MLOps pipeline that operationalizes the machine learning model beyond experimentation. Data versioning using **DVC** ensures dataset traceability and reproducibility, while scalable preprocessing with **Apache Spark** supports high-volume text data. **MLflow** enables systematic experiment tracking, metric logging, and model registry management, facilitating objective model selection and governance. **GitHub Actions** automates CI/CD workflows, ensuring consistent validation and deployment of model updates. Model serving via **FastAPI**, containerized using **Docker**, and deployed on **Render**, enables low-latency real-time inference. Post-deployment, **Evidently AI** monitors data and prediction drift, while **SHAP** provides explainable insights into model decisions. Collectively, these tools ensure scalability, automation, explainability, and continuous performance monitoring, demonstrating a production-ready and ethically governed MLOps life cycle.



METHODOLOGY

Go, change the world

• Data Collection:

Raw textual data is collected from online communication platforms such as social media and discussion forums. The data is unstructured and includes slang, spelling variations, emojis, and informal language.

• Exploratory Data Analysis (EDA):

EDA is performed to understand dataset characteristics, including class distribution (toxic vs. non-toxic), frequently occurring words, noise patterns, and data imbalance. This analysis guides preprocessing and modeling decisions.

• Data Cleaning and Labeling:

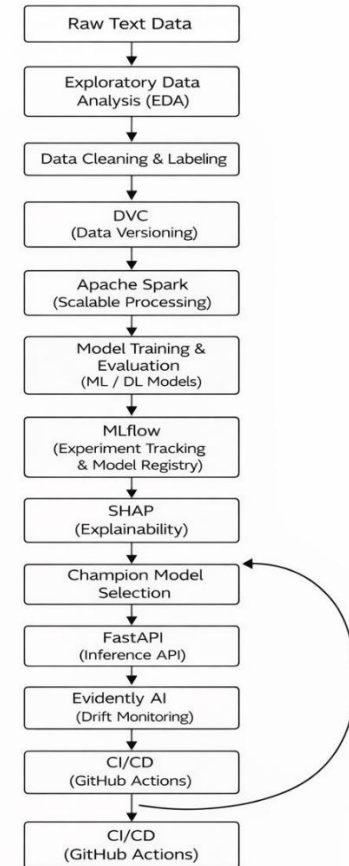
Text preprocessing involves removing URLs, symbols, repeated characters, and extra spaces, along with standardizing text formats. Each comment is labeled as toxic or non-toxic for supervised learning.

• Data Versioning with DVC:

Data Version Control (DVC) is used to track dataset versions, ensuring reproducibility, experiment consistency, and traceability across data updates.

• Scalable Processing with Apache Spark:

Apache Spark enables distributed and parallel processing of large-scale text data, improving the efficiency of preprocessing and feature extraction.





- **Model Training and Experiment Tracking:**

Multiple machine learning and deep learning models are trained on the processed dataset. MLflow is used to log parameters, metrics, configurations, and artifacts for systematic experiment comparison.

- **Explainable AI using SHAP:**

SHAP is integrated to explain individual model predictions by quantifying the contribution of each word or feature, improving transparency, interpretability, and trust.

- **Champion Model Selection:**

Models are evaluated using accuracy, precision, recall, and F1-score. The best-performing and most reliable model is selected as the Champion Model for deployment.

- **Model Deployment with FastAPI:**

The Champion Model is deployed using FastAPI to provide real-time toxicity prediction through RESTful API endpoints.

- **Post-deployment Monitoring with Evidently AI:**

Evidently AI continuously monitors data drift and prediction behavior, detecting performance degradation caused by evolving language or user behavior.

- **CI/CD Automation:**

CI/CD pipelines automate model retraining, testing, and redeployment when drift or performance issues are detected, ensuring long-term system reliability and scalability.

IMPLEMENTATION

Go, change the world

• **Modular Python-based Development:**

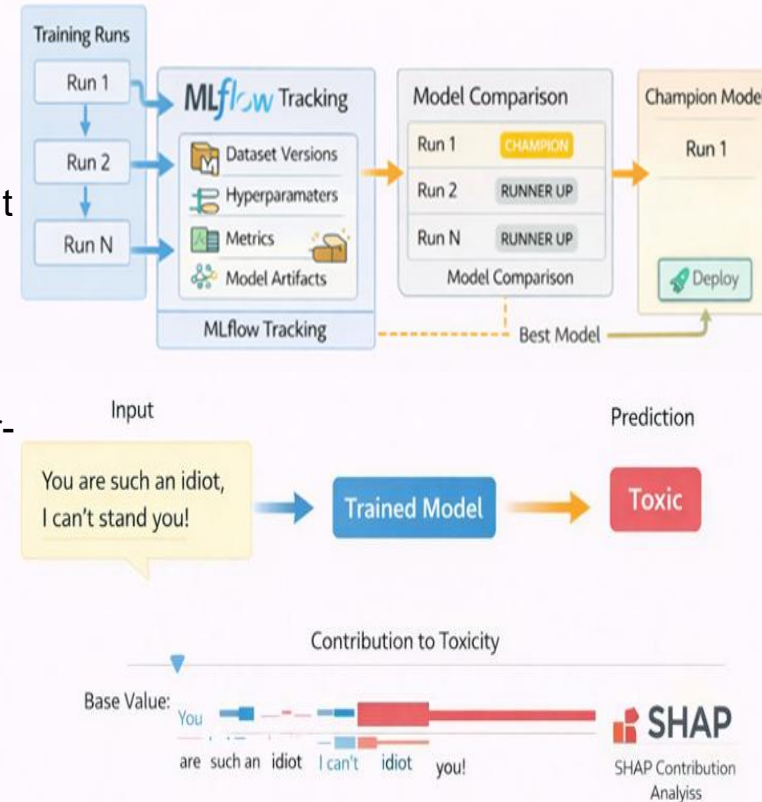
The system was implemented in Python using a modular architecture, enabling independent development of data processing, model training, explainability, serving, and deployment components.

• **Progressive Model Training:**

Multiple models were trained using Scikit-learn and TensorFlow, progressing from traditional baselines (Logistic Regression, Random Forest) to deep learning models (LSTM and Transformer-based) for improved contextual understanding.

• **Experiment Tracking and Model Selection:**

MLflow was used to track hyperparameters, metrics, artifacts, and model versions, enabling systematic comparison and selection of the best-performing **Champion Model**.





IMPLEMENTATION

Go, change the world

• Explainable AI Integration:

SHAP was incorporated to explain individual predictions by highlighting word-level contributions, improving transparency, trust, and ethical accountability.

• Real-time Model Serving:

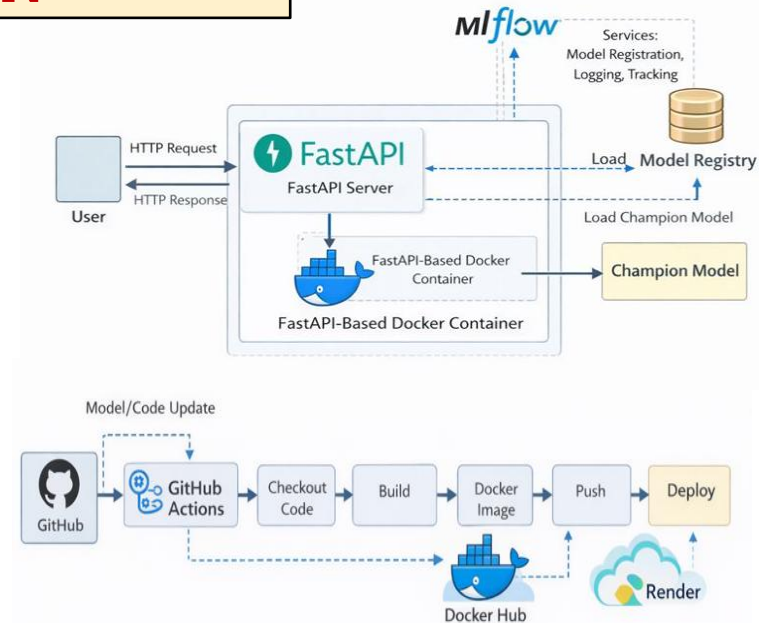
The Champion Model was deployed as a low-latency REST API using FastAPI, allowing seamless integration with chat platforms and dashboards.

• Automated Deployment and Scalability:

Docker containerization and GitHub Actions–based CI/CD pipelines enabled consistent environments, automated testing, and rapid deployments.

• Cloud Hosting:

The containerized FastAPI service was deployed on Render, providing scalable and reliable access to the moderation system in production.



Tool	Role
Docker	Containerization
GitHub Actions	CI/CD automation
Render	Cloud hosting



REFERENCES

- [1] P. Fortuna and S. Nunes, “A survey on automatic detection of hate speech in text,” *ACM Computing Surveys*, vol. 55, no. 2, pp. 1–37, 2022.
- [2] T. Davidson, D. Warmesley, M. Macy, and I. Weber, “Automated hate speech detection and the problem of offensive language,” in *Proc. Int. Conf. Web and Social Media (ICWSM)*, 2022, pp. 512–515.
- [3] A. Schmidt and M. Wiegand, “A survey on hate speech detection using natural language processing,” *Social Network Analysis and Mining*, vol. 13, no. 1, pp. 1–23, 2023.
- [4] Z. Zhao, J. Gao, and X. Huang, “Mitigating identity term bias in toxic comment classification,” *Information Processing & Management*, vol. 59, no. 6, 2022.
- [5] S. Risch, A. R. Arora, and R. Krestel, “Transformer-based approaches for abusive language detection,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 789–799.
- [6] A. Vidgen, D. Bontcheva, and L. K. Derczynski, “Challenges and frontiers in abusive content detection,” *Computational Linguistics*, vol. 49, no. 3, pp. 547–580, 2023.
- [7] Y. Cao, L. Wang, and M. Li, “Beyond classification: Understanding real-world toxic content moderation workflows,” in *Proc. ACL*, 2024, pp. 1021–1033.



THANK YOU